

OSSP

PRACTICAL-4

K. SHASHANK REDDY

2520030215

SEC -08

```
mrshashank@LAPTOP-203MS  ×  +  ▼
mrshashank@LAPTOP-203MS5J7:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

mrshashank@LAPTOP-203MS5J7:~$ nano wait_demo.c
mrshashank@LAPTOP-203MS5J7:~$ gcc wait_demo.c -o wait_demo
mrshashank@LAPTOP-203MS5J7:~$ ./wait_demo wait

Parent: PID = 537
Child 1: PID = 538
Child 3: PID = 540
Child 2: PID = 539
wait(): Collected child PID 540
wait(): Collected child PID 539
wait(): Collected child PID 538
All child processes completed.
mrshashank@LAPTOP-203MS5J7:~$ |
```

```
GNU nano 8.7.1 wait_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main(int argc, char *argv[]) {
    pid_t pids[3];
    int status;

    if (argc != 2 ||
        (strcmp(argv[1], "wait") != 0 &&
         strcmp(argv[1], "waitpid") != 0)) {
        printf("Usage: %s wait | waitpid\n", argv[0]);
        return 1;
    }

    for (int i = 0; i < 3; i++) {
        pids[i] = fork();

        if (pids[i] == 0) {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            fflush(stdout);
            sleep(3 - i);
            exit(i + 1);
        }

        if (pids[i] < 0) {
            perror("fork");
            return 1;
        }
    }

    printf("\nParent: PID = %d\n", getpid());

    if (strcmp(argv[1], "wait") == 0) {
        for (int i = 0; i < 3; i++) {
            pid_t pid = wait(&status);
            printf("wait(): Collected child PID %d\n", pid);
        }
    } else {
        for (int i = 0; i < 3; i++) {
            pid_t pid = waitpid(pids[i], &status, 0);
            printf("waitpid(): Collected child PID %d\n", pid);
        }
    }

    printf("All child processes completed.\n");
    return 0;
}
```

```
mrshashank@LAPTOP-203MS5J7:~$ ./wait_demo waitpid
```

```
Child 1: PID = 557
```

```
Child 2: PID = 558
```

```
Parent: PID = 556
```

```
Child 3: PID = 559
```

```
waitpid(): Collected child PID 557
```

```
waitpid(): Collected child PID 558
```

```
waitpid(): Collected child PID 559
```

```
All child processes completed.
```

```
mrshashank@LAPTOP-203MS5J7:~$ |
```

```
|      ~~~~~
```

```
Parent process PID = 593
```

```
Child PID = 594
```

```
Parent sleeping for 30 seconds...
```

```
Child process running. PID = 594
```

```
Child process exiting...
```

```
mrshashank@LAPTOP-203MS5J7:~$ ps -o pid,ppid,stat,cmd -p 2301
```

```
  PID    PPID STAT CMD
```

```
mrshashank@LAPTOP-203MS5J7:~$   PID    PPID STAT CMD
```

```
  2301   2300  Z    [zombie] <defunct>
```

```
PID: command not found
```

```
-bash: syntax error near unexpected token `newline'
```

```
mrshashank@LAPTOP-203MS5J7:~$ waitpid(pid, NULL, 0);
```

```
-bash: syntax error near unexpected token `pid,'
```

```
mrshashank@LAPTOP-203MS5J7:~$ |
```